

# 한국어 챗봇 Q→A 응답생성 프로젝트 — 진행 로그

작성 기준일: 2026-09-18 (누적 기록, 01~05번 노트북)

## 2026-09-17

- 프로젝트 착수: lecture/NLP\_수업정리.ipynb(15개 섹션, 220셀) 검토 — 토큰화, 서브워드, Word2Vec/FastText, RNN/CNN, 텍스트 증강, Transformer seq2seq(\$11), 사전학습 LM 활용(GPT-2/BERT/BART/T5/ELECTRA, \$12~15) 확인
- 데이터셋 확인: dataset/ChatbotData.xlsx — Q/A/label 11,823건 (label 0=일상 5290 / 1=이별(부정) 3570 / 2=사랑 2963)
- 아키텍처 결정 (사용자 승인): 베이스라인 = 처음부터 학습하는 인코더-디코더 Transformer (수업 \$11 구조), 전이학습(고도화) = 사전학습 인코더-디코더(KoBART/KE-T5) 파인튜닝 (수업 \$13/\$15 구조)
- 개발환경 구성: Python 3.11 가상환경(.venv, uv 사용) 생성 — 전역 Python 3.14에서는 gensim 등 일부 패키지의 Windows용 사전빌드 wheel이 없어 3.11로 전환
- GPU: NVIDIA RTX 5060 Laptop(VRAM 8GB, Blackwell) 인식 확인 — torch를 CUDA 12.8 wheel로 설치해 GPU 사용 가능하도록 구성
- 설치 패키지: torch(cu128), transformers, datasets, evaluate, gensim, sentencepiece, tokenizers, konlpy, jamo, nltk, spacy, Korpora, rouge-score, sacrebleu, nbformat/nbclient/ipykernel
- 01\_data\_eda.ipynb 작성 및 실행: 결측치 0건, (Q,A) 완전 중복 73건 확인 후 제거(11,823→11,750건), label 분포 확인, Q/A 길이 분포(95 percentile: Q 25음절 / A 27음절) 확인, train/val/test = 9,400/1,175/1,175건으로 분할(label 비율 유지)·저장

## 2026-09-18

- 01\_data\_eda.ipynb 수정 (사용자 피드백 반영): (Q,A) 중복 행을 실제로 눈으로 확인하는 코드 추가(정제→중복행 표시→제거 3단계로 분리), percentile의 의미와 "왜 최댓값이 아니라 95 percentile을 max\_length 기준으로 쓰는가"에 대한 설명 보강
- 02\_tokenization\_embedding.ipynb 작성 및 실행: 어절/형태소(Okt)/음절/자모 토큰화 비교(평균 토큰 수 3.5 / 5.3 / 10.3 / 24.5), SentencePiece BPE 직접 학습(vocab 8000) 및 OOV 대응 확인, KoBART 사전학습 토큰나이저(vocab 30000)와 비교 — "파인튜닝 시엔 사전학습 모델 고유 토큰나이저를 그대로 써야 하는 이유" 확인, gensim Word2Vec 임베딩 학습(skip-gram, dim 100) 및 코사인 유사도 검증, PCA 시각화
- 03\_baseline\_transformer.ipynb 작성 및 실행: 처음부터 학습하는 인코더-디코더 Transformer(파라미터 약 601만개, 임베딩·출력층 weight tying), 30 epoch 학습(train loss 29.75→6.34, val loss 21.89→6.38, early stopping 미발동) → greedy decoding에서 "사람 사람 사람..." 같은 반복 붕괴 관찰
- 04\_finetune\_pretrained.ipynb 작성 및 실행 — Part 1(베이스라인 재실험): 03번과 동일 설정으로 epoch 상한만 30→150 확대 → 114epoch에서 조기 종료(train 3.36 / val 5.83), 반복 붕괴 해소 확인 → "에폭 부족"이 주요 원인이었음을 검증(단, val loss가 5.8대에서 정체되어 이 데이터·모델 규모의 사실상 한계로 판단)
- Part 2(KoBART 파인튜닝): gogamza/kobart-base-v2 로드(파라미터 약 1.24억개, 베이스라인의 약 21배), AdamW-lr 5e-5-mixed precision으로 파인튜닝 → 7epoch에서 조기 종료(best val loss 2.79) — 베이스라인보다 훨씬 적은 학습으로 더 낮은 loss 도달(전이학습 효과 확인), 다만 train loss가 0.42까지 내려가는 동안 val loss는 4epoch째부터 반등(빠른 과적합 경향) 및 greedy 응답 일부에서 구 단위 반복 관찰
- 05\_decoding\_strategy.ipynb 작성 및 실행: greedy / beam search(k=5) / top-k-top-p sampling(top\_k=50, top\_p=0.9, temperature=0.8) 비교
- 베이스라인: greedy·beam은 반복 붕괴 해소 확인. 반대로 sampling 적용 시 오히려 반복 증가(연속 반복률 0.375, 예: "저도 슬픈 슬픈 눈.") — 모델 확신이 낮은 상태에서는 sampling이 반복을 줄여준다는 통념이 성립하지 않을 수 있음을 확인

- KoBART: greedy-beam 모두 구(phrase) 단위 반복 잔존(distinct-2 0.85~0.86, 예: "제가 있잖아요. 그럴 때가 아니죠. 제가 있잖아요..."). no\_repeat\_ngram\_size=3 추가 시 distinct-2 1.000으로 반복 해소 — beam 자체보다 반복 억제 장치의 유무가 더 결정적이었음을 확인
- 다음 단계 기본 디코딩 설정 확정: 베이스라인 = beam search(k=5), KoBART = beam search + no\_repeat\_ngram\_size=3
- PROJECT\_FLOW.md 갱신: 03/04번 실험 결과("3.5. 실험으로 확인된 것" 절)를 설계 근거 문서에 반영, 진행 상태 표 갱신

## 다음 예정

- 06\_hparam\_comparison.ipynb — 학습률/배치/에폭/임베딩 초기화(랜덤 vs Word2Vec) 등 하이퍼파라미터 비교, KE-T5 비교
- 07\_evaluation.ipynb — BLEU/ROUGE-L/Distinct-n/빈 응답 비율로 베이스라인(30ep/150ep) vs KoBART 정량 비교
- 08\_error\_analysis.ipynb — 오류 유형 분류, 대표 실패 사례 분석, 개선 전/후 비교
- 보고서(.pdf)·발표자료 작성

---

참고 문서: 설계 근거.전체 흐름은 PROJECT\_FLOW.md, 남은 작업 목록은 PLAN.md, 노트북 산출물은 notebooks/ 폴더 참고.